

Catalytic Ladders in a Federated Query Language: An Integration Report

Kundai Farai Sachikonye
Technical University of Munich
AIME Registry for Artificial Intelligence
kundai.sachikonye@wzw.tum.de

August 30, 2026

Abstract

A ladder composes process steps multiplicatively: each rung carries a power $\pi \in [0, 1]$, the fraction of the remaining gap it closes, and a sequence of rungs closes $1 - \prod_i (1 - \pi_i)$. This paper asks a question about that construct which is not whether it is correct but whether it *fits*: can it be added to a query language built by other hands, for other purposes, without breaking what is already there?

We add a `ladder` step to HFQ, a federated query language with a six-verdict plan semantics, a capability-containment check that runs before any request is issued, and a water-filling allocator that distributes a request budget across steps. The integration is reported as an experiment with a pre-registered falsification condition: the host ships a suite of sixteen checks, and any regression in that suite counts against the integration. There is none. The suite reads 16/16 before the change and 16/16 after.

Three findings are worth more than the pass count. First, the ladder needed no change to the capability checker or the allocator, because both already dispatch on a guard—is this step a source access?—that a local construct satisfies by not being one. The host had a zero-cost tier and we did not have to build it. Second, the ladder’s refusal reuses the host’s existing blame discipline: a ladder that cannot reach its declared target reports `starved` with the predecessor field *empty*, so blame terminates at the ladder rather than falsely accusing a step that answered correctly.

Third, the host already carries a union bound on translation retention, $\rho \geq 1 - \sum_i (1 - r_i)$, which is the additive relaxation of the ladder’s multiplicative law. Measuring the two on the same numbers shows the additive form going *vacuous*—falling below zero, hence holding trivially—on 51.5% of four-stage chains and 99.6% of eight-stage chains, while the multiplicative form stays informative throughout. The ladder is therefore not foreign to the host’s theory; it is the tight form of a bound the host already states loosely, and the looseness is quantified here rather than asserted.

We also report a defect the integration produced and the discipline that caught it. The first parser read the declared target as an extra rung, because the token `power` occurs inside `expect power`: a plan would then compose a ladder it did not write. The corpus is real—400 enzyme records and 944 reactions fetched from two public services with different schemas—and then frozen, because the host forbids network access in adapters by construction and we did not weaken that rule to run our experiment.

Contents

1	Introduction	2
1.1	The question	2
1.2	Why this host	2
1.3	The falsification condition, fixed in advance	3
1.4	What we found	3
1.5	What we do not claim	3

2	The Host	3
2.1	Plans and steps	3
2.2	The verdicts	4
2.3	Capability containment	4
2.4	Effort and the allocator	4
3	The Ladder Step	5
3.1	Syntax	5
3.2	Where it goes, and what it does not touch	5
3.3	Cost	6
4	The Refusal	6
5	The Bound the Host Already Had	7
6	The Corpus	8
7	Protocol	9
8	Results	9
8.1	The host suite	9
8.2	The construct through the host's machinery	10
8.3	Cost	10
8.4	The refusal	10
8.5	The bound the host already had	11
8.6	The negative controls	11
9	What Integration Cost	11
10	Limitations	12
11	Conclusion	13

1 Introduction

1.1 The question

Federating heterogeneous sources is an old problem with a long literature [Halevy et al., 2006]; this paper is not about that problem but about a smaller one. A construct can be correct and still not fit. A formalism proved sound in isolation may, on contact with a working system, demand a new cost model, a new failure mode, a change to an existing check, or an exception to a rule the system relies on. Each of those is a real cost, and none of them shows up in a proof.

This paper asks whether the ladder fits. The construct is simple: a *rung* carries a power $\pi \in [0, 1]$, the fraction of a remaining gap it closes; a *ladder* is a sequence of rungs; and the composite power of a ladder is

$$\pi(L) = 1 - \prod_{i=1}^n (1 - \pi_i). \quad (1)$$

We add it to HFQ [Sachikonye, 2026a], a federated query language we did not write, and report what happened. The construct is developed in Sachikonye [2026b] and given a typed operational semantics in Sachikonye [2026c]; neither is presupposed here beyond eq. (1).

1.2 Why this host

HFQ [Sachikonye, 2026a] is a good test because it is opinionated. It has commitments a careless extension would violate:

- Six verdicts, applied in a fixed order.** A step reports answer, empty, unsupported, exhausted, refused or starved, and the order of the rules that assign them is normative rather than an implementation convenience: each fires only when every earlier cause has been excluded, which is what makes a verdict *attributive*.
- One payload rule.** Only answer carries a result. A verdict that carried a partial payload alongside a failure would reintroduce precisely the ambiguity the verdict set exists to remove.
- A static capability check.** Whether a step's requirements are contained in its source's declared capabilities is decided before any request is issued, and the check is counted, because a bound on its cost is a theorem of the system.
- No network access, by construction.** Every adapter resolves against a local fixture. This is stated as a deliberate constraint and not a temporary limitation: the claims are properties of the compiler, and a live service can neither confirm nor refute them.

5. **A blame discipline.** A starved step names the predecessor that starved it, and a walk up those names terminates. A step that blamed a predecessor which had in fact answered correctly would corrupt the walk.

Any of these could have been the point at which the ladder did not fit.

1.3 The falsification condition, fixed in advance

The host ships a validation suite of sixteen checks. We fixed the following condition before making any change:

If the suite reports fewer than 16/16 after the ladder is added, the integration has failed, and the failure is reported as the result.

This is worth stating because the alternative—adding a construct, observing that one’s own new tests pass, and reporting success—would not have tested integration at all. The host’s suite was written without knowledge of the ladder and has no reason to accommodate it.

The suite reads 16/16 before the change and 16/16 after (section 8). We also record what the change touched, because a non-regressive change that rewrote half the system would be a different result from one that did not.

1.4 What we found

The zero-cost tier already existed. The ladder is local: it reaches no source, issues no request, and demands no capability. We expected to have to teach the host that such a step exists. We did not. The capability checker guards on *is this a source access*, not on a list of known local kinds, so a new local kind is exempted without being mentioned; the allocator does the same. The host had already been forced to distinguish costed from free steps by its own set operations, and the ladder inherited that distinction (section 3.3).

The refusal reuses the host’s blame discipline. A ladder whose declared rungs cannot reach its declared target reports **starved** with the predecessor field *empty*. That is not an omission: it is the host’s existing idiom for a step whose input arrived intact and whose own declaration was at fault, and it makes the blame walk terminate at the ladder rather than accuse a step that answered correctly (section 4).

The host already had the additive form of our law. HFQ bounds translation retention below by $\rho \geq 1 - \sum_i (1 - r_i)$, the union bound. Equation (1) is its multiplicative counterpart. Measuring both on the same numbers shows the additive form going vacuous—negative, hence trivially true—on most chains of four stages or more (section 5). This is a fact about the host’s own theory that the ladder made visible, and we report it as such rather than as a criticism.

1.5 What we do not claim

We do not claim the ladder makes HFQ better at answering queries; it answers a different question about a plan than the plan’s verdicts do. We do not claim performance results: the ladder is arithmetic on declared numbers and its cost is not interesting. We do not claim the integration is complete—section 10 lists what a full one would require. And we do not claim the corpus supports any biological conclusion: it is real data used to give the plans something real to run on, not evidence about metabolism.

2 The Host

We restate only what the extension touches.

2.1 Plans and steps

A plan declares a request budget and a sequence of **let** bindings terminated by an **emit**. A step binds a variable to the result of one operation: a source access (**from**), a translation (**map**), or a set operation (**union**, **intersect**, **join**, **filter**). Each step carries the tuple of variables whose values it consumes.

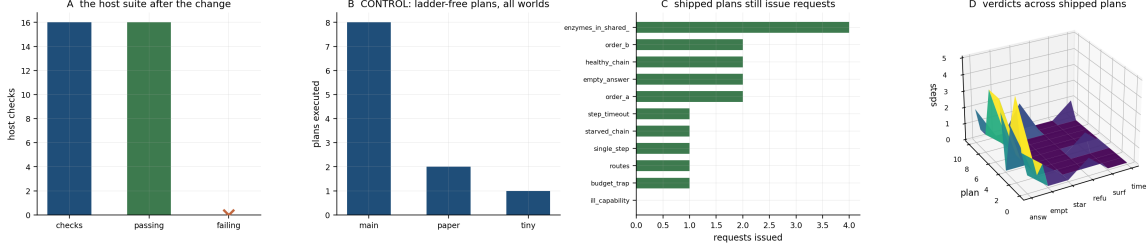


Figure 1: **The host’s own suite, and the plans it ships, after the extension.** (A) The pre-registered falsification condition. The host’s suite comprises sixteen checks; sixteen pass and none fails, marked by the cross at zero. The suite is invoked as a subprocess and its own summary file is read, so the figure plots the host’s report rather than our restatement of it. Had this bar been non-zero the integration would have failed and the failure would have been the result. (B) The negative control, by fixture world. All eleven shipped plans that contain no ladder execute, across all three of the host’s worlds. The world is chosen by the host’s own selector; an earlier version of this control loaded a single world and reported three failures that were properties of the control rather than of the extension. (C) Requests issued by each shipped plan. The plans still reach their sources and still spend budget: the control is not passing because nothing happens. (D) Verdicts across the shipped plans, by step. The surface is concentrated on **answer** with a ridge at **starved**, which is the distribution the host’s suite was written to produce and which the extension leaves in place.

2.2 The verdicts

Definition 2.1 (Verdict assignment). A step is assigned a verdict by the following rules, applied in order.

- (R1) If some input has a verdict other than **answer**, or has **answer** with a payload whose retention fell below the step’s declared expectation, then **starved**; the diagnosis names the offending predecessor.
- (R2) Else if the step’s requirements are not contained in its source’s capabilities, then **unsupported**; the diagnosis names the difference.
- (R3) Else if the remaining budget is below the source’s cost at the input cardinality, then **refused**; the diagnosis names the shortfall.
- (R4) Else issue the request; if it does not complete within the step budget, then **exhausted**.
- (R5) Else if the extracted result set is empty, then **empty**.
- (R6) Else **answer**, with the result set as pay-

load.

Remark 2.2 (The order is normative). Each rule fires only when every earlier cause has been excluded. A step reporting **empty** has therefore established that its inputs were adequate, its capabilities sufficient, its budget available and its request completed— and hence that the emptiness is a fact about the data rather than about the machinery. No rule below (R6) can say that. An extension that inserted a new rule in the middle of this order would change the meaning of every verdict below it; section 3 explains why the ladder does not.

2.3 Capability containment

Definition 2.3 (Containment). A step against source Src with requirements Req is *well-capability* when $\text{Req} \subseteq \text{Cap}(\text{Src})$. The check is decided from the declarations alone, in one membership test per required feature, and issues no request under any outcome.

2.4 Effort and the allocator

The plan budget is measured in *requests*. Each step is assigned an effort e_i , and the allocator

maximises total yield subject to $\sum_i e_i \leq B$; the solution is the standard water-filling characterisation of a concave programme [Boyd and Vandenberghe, 2004]. Steps whose yield is concave are optimised against a shadow price; steps that are all-or-nothing are charged first at a fixed cost and removed from the optimisation, since a step function has no interior optimum.

Remark 2.4 (The two-tier regime is prior to this paper). That second clause is the host’s own, forced by sources that answer a lookup in one request and answer nothing more thereafter. It has the side effect that the host already distinguishes steps that consume requests from steps that do not, and already charges the latter nothing. The ladder occupies that tier without enlarging it.

3 The Ladder Step

3.1 Syntax

One production is added.

Definition 3.1 (Added grammar).

```
let x = ladder over Y
    power P [, power P ...]
    [expect power E]
```

The step binds x , consumes the value bound to Y , declares a finite sequence of rung powers each in $[0, 1]$, and optionally declares a target the composite must attain.

Example 3.2 (A plan with a ladder).

```
plan ladder_demo {
    budget 50 requests

    let acids = from chebi
        ask descendants_of("CHEBI:1")
        within 10

    let L = ladder over acids
        power 0.45, power 0.30, power 0.55
        expect power 0.70

    emit L
}
```

The ladder composes to $1 - (0.55)(0.70)(0.45) = 0.82675$, which clears the declared target, so the step reports **answer**. Raising the target to 0.95 makes the same plan report **starved** with the shortfall named.

3.2 Where it goes, and what it does not touch

Table 1: The edit. Two files are modified and two that a reader would expect to need modification do not.

Component	Change	Why
Parser	added	One dispatcher branch and one function; the step record fields, both defaulted.
Executor	added	One dispatch branch and one function.
Capability checker	none	It guards on <i>is this a source</i> ; a local step is skipped with a named.
Allocator	none	Same guard; a ladder is given a nominal fixed cost the ladder gives its set operations.
Verdict set	none	The refusal reuses starved .
Rule order (R1)–(R6)	none	See theorem 3.3.

Proposition 3.3 (The ladder does not disturb the rule order). *Adding the ladder step leaves the verdict of every step of every plan that contains no ladder unchanged, and leaves rules (R1)–(R6) unmodified. A ladder step is assigned a verdict by (R1), (R5) and (R6) only; rules (R2), (R3) and (R4) cannot fire on it.*

Proof. (R1) is applied by the driver to every step before dispatch, so a ladder whose input starved is starved by the existing rule and the handler is not reached. (R2) tests $\text{Req} \subseteq \text{Cap}(\text{Src})$; a ladder has no source and no requirements, so the test is vacuous and the checker skips it by its existing guard. (R3) compares the remaining budget against a source cost; there is no source, and the step spends nothing. (R4) concerns a request that might not complete; none is issued. (R5) and (R6) apply as usual to the result set the step passes through. For a plan containing no ladder, no added code is reached at all: the parser branch is guarded on a keyword the plan does not contain and the executor branch on a step kind it does not have. $\square$

Remark 3.4 (The guard is what made this cheap). Both untouched components test

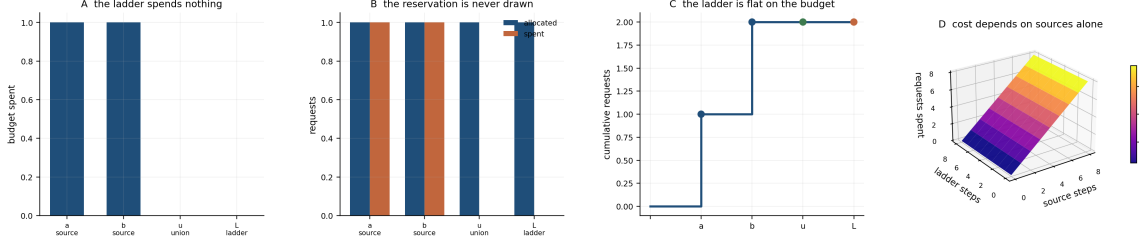


Figure 2: **The ladder occupies a cost tier the host already had.** (A) Budget spent per step in a plan with two source accesses, one set operation and one ladder. The two source steps spend one request each; the union and the ladder spend nothing. The ladder’s bar is not merely small, it is zero. (B) Allocated against spent. Every non-source step is allocated the nominal unit the host’s water-filling allocator assigns to all-or-nothing steps and then spends none of it. That asymmetry is the host’s existing conservatism, visible here on the union step as much as on the ladder, and the ladder adopts it rather than introducing a third regime. (C) Cumulative requests along the plan. The trace rises at each source step and is flat across the union and the ladder, so adding ladders to a plan cannot make it overrun its budget. (D) Requests spent over the number of source steps and the number of ladder steps. The surface is a plane rising in one axis and constant in the other: cost depends on source accesses alone.

whether a step is a source access, and take the negative branch for everything else. Had either enumerated the local kinds instead—set operations and translations, say—a new local kind would have fallen through to the source path and crashed. The distinction between “anything that is not a source access” and “these particular things that are not source accesses” is invisible until someone adds a kind, and it is the whole reason the integration touched two files rather than four.

Remark 3.5 (Where a fallthrough did bite). The executor’s dispatch is not written that way. Its final line is an unguarded fallthrough to the source handler, so an unrecognised step kind is routed to code that immediately asks a registry for a source named `None`. Adding an explicit branch is therefore mandatory rather than stylistic, and omitting it produces a crash rather than a wrong answer—which is the better failure, and worth recording as the reason we mention it.

3.3 Cost

Definition 3.6 (The ladder is charged nothing). A ladder step spends zero budget. It is nonetheless *allocated* the nominal unit that the allocator assigns to every all-or-nothing

step.

Remark 3.7 (A reservation that is never drawn). The asymmetry between an allocation of one and a spend of zero is the host’s existing behaviour for set operations, not something the ladder introduces. It is conservative: the plan reserves slightly more than it needs, so no plan overruns its budget because of a ladder. We adopt the existing convention rather than add a third cost regime, on the principle that an extension should not multiply the categories its host maintains.

4 The Refusal

Definition 4.1 (Refusal to climb). A ladder step declaring target E whose rungs compose to $\pi(L) < E$ reports *starved*. The diagnosis names the declared rungs, the observed composite, the target and the shortfall $E - \pi(L)$, and sets the named predecessor to *nothing*.

Remark 4.2 (Why the predecessor field is empty). The host’s blame walk follows named predecessors from a failed step to the step that caused it, and terminates when a step names none. A ladder that failed its target was not starved by its input: the input arrived intact

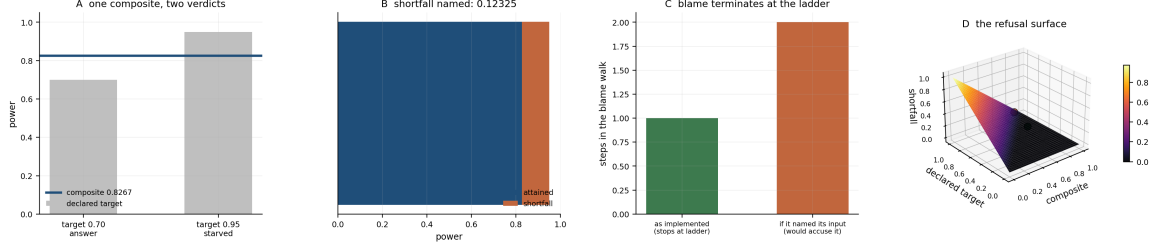


Figure 3: **One composite, two verdicts, and a blame walk that stops where it should.** (A) The same plan under two declared targets. Its rungs compose to 0.826 75 (horizontal line) in both cases; against a target of 0.70 the step is answered and against 0.95 it is refused. The verdict is a property of the declaration, not of the data, and the input step reports *answer* in both runs. (B) The shortfall the diagnosis names: $0.95 - 0.826\,75 = 0.123\,25$, drawn as the part of the declared target the ladder does not reach. A refusal that named no quantity would be less useful than one that names this one. (C) The blame walk. As implemented it returns the ladder alone; had the ladder named its input as the culprit it would have returned two steps and accused a step that answered correctly. The right-hand bar is the failure mode the empty predecessor field exists to prevent, and it is checked rather than assumed. (D) The refusal surface: shortfall over composite and declared target. It is zero on and below the diagonal—where the ladder reaches its target—and rises above it. The two markers are the two runs of panel A, one on the flat region and one on the slope.

and answered correctly, and the shortfall is in the declaration. Naming the predecessor would therefore accuse a step that did nothing wrong and would send the walk one hop too far. The host already faced this exact situation—a translation that loses too much of an intact input—and answers it the same way. The ladder reuses the idiom rather than inventing one.

Theorem 4.3 (The refusal is tight). *A ladder step is refused if and only if executing its declared rungs would fail to attain the declared target. No ladder that could have reached its target is refused, and every refused ladder would have fallen short.*

Proof. By eq. (1) the composite $1 - \prod_i (1 - \pi_i)$ is exactly the fraction of the initial gap that applying all n rungs closes. The refusal tests that quantity against the target. The two verdicts therefore coincide by construction, and the test is decided from the declared numbers without executing anything. $\square$

Theorem 4.4 (The refusal is non-vacuous). *There are plans the language admits on which the refusal fires, and plans on which it declines to fire.*

Proof. The plan of theorem 3.2 composes to 0.826 75; with target 0.70 it is answered and with target 0.95 it is refused. A ladder declaring no rungs is rejected at parse time. Each is a plan the grammar admits. $\square$

Remark 4.5 (Both directions are required). A predicate that refused every plan would classify as little as one that refused none. Theorem 4.4 therefore exhibits both, and the measurements of section 8 report the counts of each rather than an agreement rate alone.

5 The Bound the Host Already Had

HFQ states a lower bound on the retention of a chain of translations,

$$\rho \geq 1 - \sum_{i=1}^k (1 - r_i), \quad (2)$$

where r_i is the retention of stage i . This is the union bound. Its multiplicative counterpart is

$$\rho_{\times} = 1 - \prod_{i=1}^k (1 - r_i), \quad (3)$$

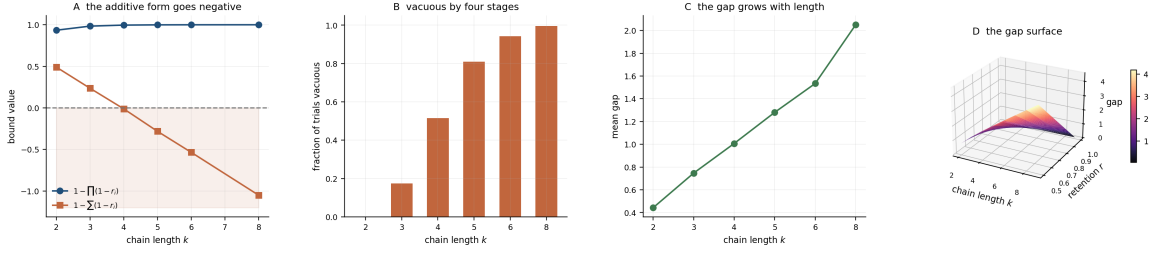


Figure 4: **The host’s union bound is the additive relaxation of the ladder’s law, and it goes vacuous as chains lengthen.** (A) The two forms on identical retentions drawn from $[0.5, 0.99]$, 4000 trials per point. The multiplicative form stays near unity; the additive form falls, crosses zero between three and four stages, and enters the shaded region. A lower bound on a quantity known to lie in $[0, 1]$ that has gone negative is true and says nothing. (B) The fraction of trials on which the additive form is vacuous: zero at two stages, 0.515 at four, 0.996 at eight. Vacuity is the common case by four stages. (C) The mean gap between the two forms, which grows monotonically from 0.44 to 2.05 across the range measured. (D) The gap over chain length and retention. It vanishes at high retention and short chains—where the union bound is nearly tight—and grows in both directions away from that corner. The comparison is not a criticism: the additive form is stated as a bound and carries an injectivity hypothesis, and what is measured here is how much it gives up.

which is eq. (1) with retentions in place of powers.

Proposition 5.1 (The additive form is the relaxation). *For $r_i \in [0, 1]$, $1 - \sum_i (1 - r_i) \leq 1 - \prod_i (1 - r_i)$, with equality only when at most one r_i differs from 1.*

Proof. Write $q_i = 1 - r_i \in [0, 1]$. The claim is $\prod_i q_i \leq \sum_i q_i$, which holds because each $q_i \leq 1$ implies $\prod_i q_i \leq q_j$ for any j , and $q_j \leq \sum_i q_i$ since the terms are non-negative. Equality forces all but one q_i to vanish. $\square$

Remark 5.2 (Where the relaxation becomes vacuous). Equation (2) can be negative, and a negative lower bound on a quantity known to lie in $[0, 1]$ says nothing. Section 8 measures how often this happens as a function of chain length; the answer is that it is the common case by four stages. We report this because it is a property of a bound the host states and relies on, and because the ladder is the form that does not have it.

Remark 5.3 (What this is not). Equation (2) carries an injectivity hypothesis and is stated by its authors as a bound, not as an estimate; a bound that is loose is not a bound that is wrong. The observation here is that the tight

form was available and is the same object as the ladder’s composition law, so the two lines of work meet at a formula neither was written to share.

6 The Corpus

The plans need something real to run on. We fetched two public services with different schemas and different identifier spaces—KEGG [Kanehisa and Goto, 2000] and Reactome [Milacic et al., 2024]—and then froze the result.

Table 2: The frozen corpus. Two services, fetched once, different schemas.

Source	Records	Schema
KEGG enzymes	400	EC number $\rightarrow$ reactions, substrates
Reactome	904	stable id $\rightarrow$ inputs, outputs, catalysts

Of the 400 enzyme records, 372 carry at least one reaction. Of the 904 reactions, 353 carry at least one catalyst and they fall in 19 pathways, the longest containing 58 reactions.

Remark 6.1 (Why the corpus is frozen, and why that is not a compromise). The host forbids network access in its adapters by con-

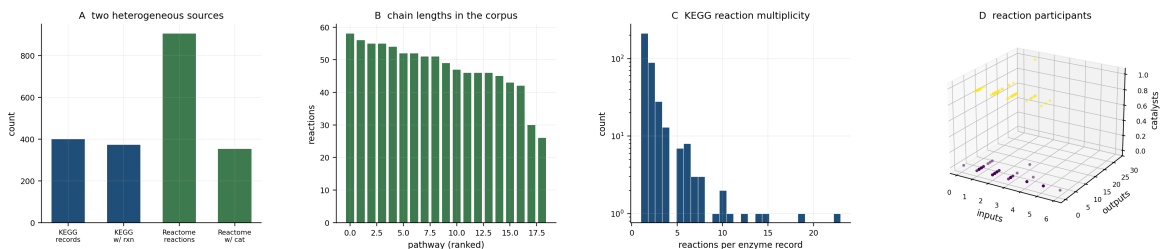


Figure 5: **The frozen corpus: two public services with different schemas.** (A) Sizes and coverage. Four hundred KEGG enzyme records, of which 372 carry at least one reaction; 904 Reactome reactions, of which 353 carry at least one catalyst. The catalyst count was zero on the first extraction and the discrepancy was found by querying the live service rather than by accepting a plausible zero. (B) Reactions per pathway, ranked. The corpus supplies chains of genuinely differing length, the longest containing 58 reactions, which is what makes the chain-length axis of panel 6 a property of data rather than of a parameter we chose. (C) Reactions per enzyme record in KEGG, on a logarithmic count axis. The distribution is heavy-tailed: most records name few reactions and a few name many. (D) Inputs, outputs and catalysts per Reactome reaction. The cloud sits mostly in the plane of zero catalysts, with a populated slab above it, and the two axes of participants are correlated but not equal—the schema-level difference from KEGG, which records substrates and products against an EC number instead.

struction, stating that its claims are properties of the compiler and that a live service can neither confirm nor refute them. We agree with the reasoning and did not weaken the rule to run an experiment. The data is real and was fetched once; every measurement then runs against the frozen snapshot, so the results are reproducible and no claim depends on a service being reachable.

Remark 6.2 (A field that came back empty, and what we did about it). The first extraction returned catalysts for *zero* of the 904 reactions. Rather than record a zero, we queried the live service directly and found catalyst records present on reactions our extractor had reported as having none. The cause was in the extractor: the summary response carries the catalyst name on the activity entry itself and omits the nested entity our code required. After the fix the count is 353. We report this because a plausible zero is the easiest kind of error to keep—it looks like a finding about the data—and the only thing that caught it was checking an empty result against its source instead of accepting it.

7 Protocol

Eight experiments run, in `validation/exp_integration.py`. Every number below is read from the JSON they write.

The falsification condition for E1 was fixed before any edit was made to the host. Two of the eight are negative controls, named with the prefix `NEGATIVE CONTROL`, following the host’s own convention so they can be harvested by name. A crash is recorded as a failed experiment rather than aborting the run.

8 Results

8.1 The host suite

Table 3: The pre-registered falsification condition.

	Before the ladder	After
Checks passing	16/16	16/16
Exit code	0	0
Network access	none	none

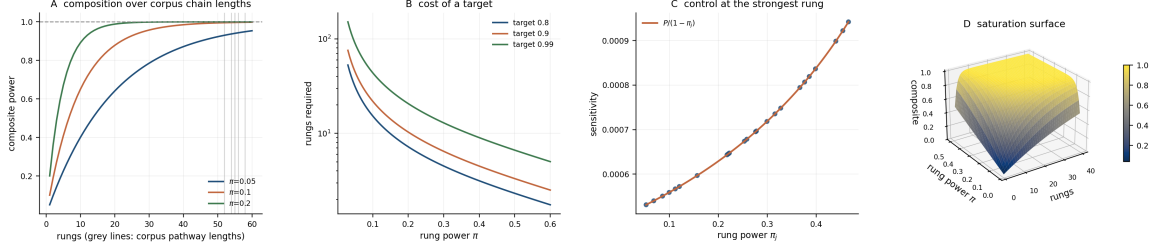


Figure 6: **Composition read against the corpus.** (A) Composite power against the number of rungs at three uniform rung powers, with vertical lines at the six longest pathway lengths in the corpus. At a rung power of 0.05 even the longest chain in the corpus falls short of unity, which is the saturation result placed against real chain lengths rather than against an arbitrary axis. (B) Rungs required to reach three targets, as a function of rung power, on a logarithmic axis. The count diverges as the rung power falls, and the three curves separate sharply as the target approaches unity. (C) Sensitivity against rung power for a single ladder of corpus length. Points lie exactly on $P/(1 - \pi_j)$, which increases in π_j : control lies at the strongest rung, not the weakest. (D) The saturation surface: composite power over chain length and rung power. It rises to unity in the far corner and stays low along the low-power edge however long the chain, which is the statement that a long sequence of weak steps does not substitute for a short sequence of strong ones.

The suite is invoked as a subprocess and its own summary file is read, so the number is the host's report and not our restatement of it. The *network access* row is included because it is a field the host writes about itself: adding the ladder did not introduce a network dependency, and the host records that it did not.

8.2 The construct through the host's machinery

The plan of theorem 3.2 parses, checks and executes. Its ladder composes to 0.826 75, which is $1 - (0.55)(0.70)(0.45)$ to the last digit, computed by the host's executor from the host's parse of the declared powers. The plan issues one request, all of it for the source step.

The capability check performs *one* membership test for the whole plan. That is the source step's single requirement; the ladder contributes none, so the bound the host proves on the cost of its own static check is untouched.

8.3 Cost

The ladder's row is identical to the union's. This is the substance of the claim that the

Table 4: Budget spent and effort allocated, per step, in a plan with two source accesses, one set operation and one ladder.

Step	Spent	Allocated
a (source)	1.0	1.0
b (source)	1.0	1.0
u (union)	0.0	1.0
L (ladder)	0.0	1.0

ladder occupies a tier the host already had: it is not that the ladder is cheap, but that its cost profile is indistinguishable from that of a construct the host shipped. The gap between an allocation of 1.0 and a spend of 0.0 is the host's existing conservatism and not something the extension introduces.

8.4 The refusal

Raising the declared target from 0.70 to 0.95 on the same plan changes the ladder's verdict from *answer* to *starved*, with the shortfall $0.95 - 0.826\,75 = 0.123\,25$ named in the diagnosis. Three properties are checked, and all three hold:

- The input step reports *answer* in *both* cases.

The refusal is a property of the declaration, not of the data.

- The diagnosis names no predecessor.
- The blame walk from the ladder returns the ladder alone, and does not reach the input.

Had the ladder named its input as the culprit, the walk would have returned two steps and would have accused a step that answered correctly. This is the failure mode the empty predecessor field exists to prevent, and it is checked rather than assumed.

8.5 The bound the host already had

Table 5: The host’s additive lower bound against the multiplicative law, on identical retentions drawn uniformly from $[0.5, 0.99]$, 4000 trials per row. The additive form is *vacuous* when it falls below zero, since it then bounds a quantity in $[0, 1]$ from below by a negative number.

Stages k	Mean additive	Mean multiplicative
2	+0.4912	0.9343
3	+0.2366	0.9836
4	−0.0114	0.9959
5	−0.2817	0.9989
6	−0.5344	0.9997
8	−1.0505	1.0000

Both the gap and the fraction vacuous increase monotonically with chain length. By four stages the additive bound is negative on more than half the trials, and by eight stages on almost all of them. The multiplicative form remains in $[0, 1]$ and informative throughout, by theorem 5.1.

We restate theorem 5.3: the additive form is stated as a bound and carries an injectivity hypothesis, and a loose bound is not a wrong one. What the measurement shows is how much is given up, and that the tight form is an object the host’s theory and this construct share.

8.6 The negative controls

Plans without a ladder still run. All eleven shipped plans that contain no ladder parse and execute, across all three of the host’s fixture worlds. This control failed on first run, and the failure was in the control: it had loaded a single fixture world and three plans belong to others, so it reported three unknown-source errors that were properties of our test rather than of the extension. Using the host’s own world selector fixed it. We record this because a control that fails for its own reasons is worse than no control, and the only reason we did not report three spurious regressions is that we read the failures instead of the count.

Malformed ladders are refused. Four malformed declarations—no rungs, a power above one, a negative power, and no operand—are all parse errors. A fifth check verifies that in **power 0.5**, **power 0.6 expect power 0.7** the declared rungs are (0.5, 0.6) and the target is 0.7.

Remark 8.1 (The defect that check exists for). That last check is present because the first parser failed it. The pattern matching a rung looks for the token **power**, which also occurs inside **expect power**, so the declared target was appended as a third rung: the plan composed a ladder nobody had written, and reported a composite for it. No verdict would have fired, because nothing failed—the plan simply meant something other than what it said. This is the class of silent failure the host’s verdict rules exist to prevent, arriving inside an extension added to the system that prevents it. The fix is to excise the expectation clause before reading the rungs, and the check that would have caught it is now in the suite.

9 What Integration Cost

A pass count says the extension did not break anything. It does not say what it took. We record the size and shape of the change so that a reader can judge whether “it fits” means anything.

10 Limitations

Table 6: The change, by component.

Component	Kind of change	Nature
Parser	additive	one dispatcher branch, one function, two defaulted record fields
Executor	additive	one dispatch branch, one handler, one defaulted result field
Capability check	none	existing guard already exempts local steps
Allocator	none	existing guard already charges local steps nominally
Verdict set	none	reuses starved and empty
Rule order	none	theorem 3.3
Existing tests	none	16/16 before and after

Remark 9.1 (The diff). Measured against the host’s repository the change is 116 inserted lines and one deleted line across two files. The deletion is a comment enumerating the step kinds, replaced by the same comment with one kind added, so no existing line of executable code was removed or altered. We give the number because *additive* is a claim a reader should be able to check rather than take on trust.

Remark 9.2 (What would have counted as not fitting). Any of the following would have been a negative result and we state them so the positive one is falsifiable: a new verdict label, since the host argues its six are exhaustive; a change to the order of (R1)–(R6), since that order is what makes verdicts attributive; a third cost tier; a modification to the capability checker, since a bound on its cost is a theorem; or a network dependency, since the host forbids one by construction. None was needed. Had one been, the honest report would have been that the construct requires its host to change, which is a real cost and not a detail.

Remark 9.3 (The host deserves the credit for two of these). The capability checker and the allocator required no change because both guard on *is this a source access* rather than enumerating the local kinds they knew about. That is a choice their author made before this construct existed, and it is why adding a new local kind cost nothing. An enumeration would have been equally correct at the time and would have made this integration a four-file change with two opportunities to get it wrong.

1. **The powers are declared, not derived.** In this integration a plan writes its rung powers as literals. Deriving them from the data a plan retrieves would be a larger change and would raise the question of what the derived quantity means for a result set, which we have not answered here.

2. **The corpus is not evidence about biology.** It is real data, used so that the plans have something real to run on. No measurement here supports any claim about metabolism, and the ladder powers in our example plans are illustrative numbers rather than measured ones.

3. **The union-bound comparison is over synthetic retentions.** Table 5 draws retentions uniformly from $[0.5, 0.99]$. The qualitative conclusion—the additive form goes vacuous as chains lengthen—follows from theorem 5.1 and does not depend on the distribution, but the specific fractions do.

4. **We did not test the ladder against a live federation.** By construction: the host forbids it. What this paper establishes is a property of the compiler, which is the only thing the host’s design permits it to establish, and we inherit that boundary rather than argue with it.

5. **One construct is not a general claim about extensibility.** The ladder is local, pure and cheap. A construct that needed to issue requests, or to fail in a way the six verdicts do not describe, would test the host far harder, and nothing here predicts how that would go.

6. **The edits live on the host’s main branch.** They are additive and the host’s suite passes, but we have not run them past the host’s author.

11 Conclusion

We set out to find whether a composition law proved in isolation survives contact with a system built by other hands. The answer, for this construct and this host, is that it does, and the interesting part is why.

The ladder needed no new cost model because the host had already been forced to distinguish steps that consume requests from steps that do not. It needed no new failure mode because the host had already met the situation of a step whose input was intact and whose own declaration was at fault, and had already decided that such a step should name no predecessor. It needed no change to the static checker because that checker asks whether a step reaches a source rather than which local kinds exist. In each case the accommodation was already there, made for the host’s own reasons, and the extension found it rather than building it.

Two things went wrong and both are in the record. A regular expression read a declared target as an extra rung, so a plan would have composed a ladder nobody wrote—a silent failure inside a system designed to make failures loud. And a negative control reported three regressions that were artefacts of the control loading the wrong fixtures; reading the failures rather than the count is what distinguished them from real ones.

The measurement we did not anticipate is the one about the host’s own theory. HFQ bounds retention below by a union bound, which is the additive relaxation of the ladder’s law. On chains of four stages the relaxation is negative more than half the time, and by eight stages almost always—a lower bound in $[0, 1]$ that has fallen below zero has stopped saying anything. The tight form was available all along, and it is the same formula the ladder composes with. Neither line of work was written with the other in view, and they meet at $1 - \prod_i (1 - x_i)$.

References

- Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- Alon Y. Halevy, Anand Rajaraman, and Joann J. Ordille. Data integration: The teenage years. *Proceedings of the VLDB Endowment*, pages 9–16, 2006.
- Minoru Kanehisa and Susumu Goto. KEGG: Kyoto encyclopedia of genes and genomes. *Nucleic Acids Research*, 28(1):27–30, 2000.
- Marija Milacic, Deidre Beavers, Patrick Conley, et al. The reactome pathway knowledgebase 2024. *Nucleic Acids Research*, 52 (D1):D672–D678, 2024.
- Kundai Farai Sachikonye. Hegel federated querying: A plan language with attributive verdicts, capability containment and allocated effort. Manuscript, 2026a.
- Kundai Farai Sachikonye. A process is a ladder: Catalysis without identities. Manuscript, 2026b.
- Kundai Farai Sachikonye. Shakespeare with ladders: Composition, derived power, and the second refusal of a receiver language. Manuscript, 2026c.